

# s5.04 — WC3 replays site — 25-minute system design draft

Supporting reference s5.04 in [the bank](#); not a ranked question. Business-style warm-up draft.

## Index

1. Time budget
2. Why this is hard (the crux)
3. Assumptions & clarifying questions
4. Back-of-envelope
5. High-level design
6. API
7. Data model
8. Deep dive A — replay identity & dedup (1v1 → 4v4)
9. Deep dive B — "today" listing, pagination, caching
10. Failure modes
11. Pop-up questions
12. Tradeoffs & wrap-up
13. References

## 1. Time budget

clarify 3 | crux 2 | BoE 3 | HLD 6 | API+schema 4 | deep dives 5 | wrap 2 = 25 min

## 2. Why this is hard (the crux)

100M readers → one identical "today" page → ~1K new replays/day  
(read-heavy, shared view) (write-trivial, but duplicated 2–8×)

- Naive design: one Postgres, `SELECT * ORDER BY played_at DESC OFFSET n`, files on local disk.
  - Breaks first on **reads**: 100M DAU hitting the same list ≈ 10K+ QPS on one DB.
  - Second break: **duplicate uploads** — a 4v4 game has 8 clients, each holding the same replay.
- The write side is tiny. This is **not** a partitioning problem.
  - 🐛 "This is a cache problem at 10K QPS, because everyone reads the same list; and an idempotency problem on writes, because every game arrives 2–8 times."
- Cost of being wrong:
  - Stale list by 30 s: nobody cares (eventual is fine).
  - Duplicate replay shown twice, or wrong winner: users notice → dedup and metadata must be **strongly consistent** (one row per game).

## 3. Assumptions & clarifying questions

Q → assumed answer → design consequence

| Question                                        | Assumed                                                                           | Consequence                                                        |
|-------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Who uploads?                                    | Top-100 players (client auto-upload) or the ladder system                         | Auth = API key / player login; upload volume bounded               |
| Game modes?                                     | 1v1 default, up to 4v4                                                            | 2–8 uploads per game → dedup on game identity, not upload          |
| Are replay files byte-identical across clients? | <b>Unknown — must verify.</b> WC3 .w3g action stream is shared, header may differ | Dedup on parsed game fingerprint, content hash as secondary        |
| Replay size?                                    | ~0.5–2 MB (assume, order of magnitude)                                            | Presigned direct-to-S3 upload; CDN for download                    |
| Default page?                                   | Latest replays, newest first, "today" grouping backfills from yesterday           | Keyset pagination on (played_at, id); day grouping is a UI concern |
| Consistency on list?                            | 30 s stale OK                                                                     | Cache listing at CDN + Redis                                       |
| Search/filter?                                  | Extended: by player, map, mode. Non-goal: full-text                               | Indexes on player_id, mode; no OpenSearch in MVP                   |
| Non-goals                                       | Live streaming, in-browser replay playback, comments                              |                                                                    |

## 4. Back-of-envelope

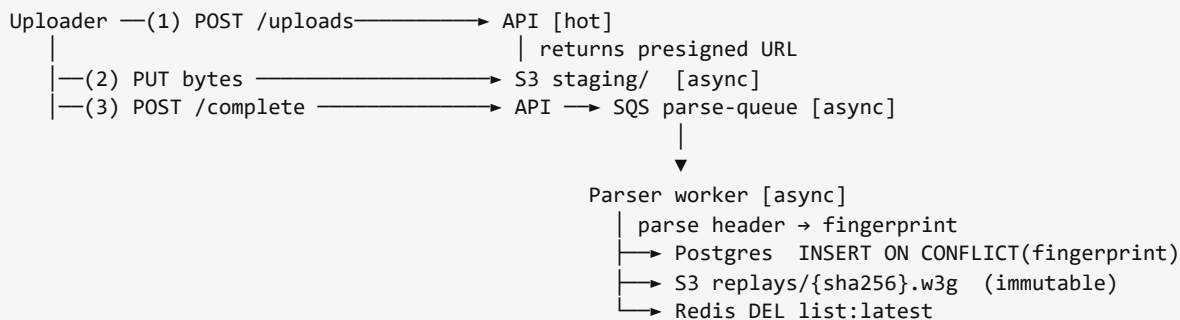
writes: 100 players × 10 games × (2..8 uploads) ≈ 1K games/day, ≤8K upload attempts/day  
reads: 100M DAU × 3 list views ≈ 300M/day ≈ 3.5K QPS avg, ~10K QPS peak (3× assume)

- Storage  
→ 1K replays × 1 MB = **1 GB/day** → **~365 GB/yr**. Trivial. S3 Standard, no tiering needed for years.
- Downloads  
→ assume 10% of DAU download 1 replay → 10M/day × 1 MB = **10 TB/day egress**.  
→ This is the real cost line → **CDN in front of S3**, immutable objects, Cache-Control: max-age=1y.
- Metadata DB  
→ 1K rows/day → a single Postgres with 2 read replicas is oversized. Bottleneck is never the DB if the cache works.
- Hot set  
→ today's list ≈ 1K rows × ~500 B JSON = **0.5 MB**. The entire day fits in one Redis value.
- 🧠 "Three numbers drive this: 10K list QPS, 10 TB/day egress, 0.5 MB hot set. So: CDN + Redis for reads, S3 + Postgres for truth."

## 5. High-level design

WRITE PATH (~1K games/day, arrives 2-8x)

=====



READ PATH (~10K QPS peak, everyone reads the same feed)

=====



- **API service [hot path]** — stateless Spring Boot on ECS; two routes: `/uploads` (writes) and `/replays` (reads). Split into two services only when scaling needs diverge; not in MVP.
- **S3 [async storage]** — `staging/` for raw uploads (24 h lifecycle delete), `replays/{sha256}.w3g` for accepted, content-addressed, immutable.
- **SQS + parser workers [async]** — parse `.w3g` header, compute fingerprint, dedup, write metadata. Decouples upload latency from parsing; retries for free.
- **Postgres [truth]** — one row per game, unique on fingerprint. Read replicas for non-cached queries.
- **Redis [hot path]** — `list:latest` (whole recent window as one sorted blob) + `replay:{id}`.
- **CDN [hot path]** — caches `GET /v1/replays` responses for 30 s and serves replay bytes from S3 forever.

### Write path (numbered)

1. Client POST `/v1/uploads {sha256, size_bytes}` with `Idempotency-Key` — API returns `upload_id` + presigned S3 PUT URL (sync). If `sha256` already accepted → return existing `replay_id`, done.
2. Client PUTs bytes to `staging/{upload_id}` — after 1. If it fails, client retries; nothing server-side changed.
3. Client POST `/v1/uploads/{upload_id}/complete` — API verifies object exists (HEAD), enqueues `ParseReplay{upload_id}` to SQS, returns `202`. If enqueue fails → `500`, client retries step 3 (idempotent).
4. Worker receives message, downloads staging object, validates magic bytes + version — on failure → mark upload `rejected`, delete staging object, ack message (terminal, not retryable).
5. Worker parses header → `fingerprint`, `players`, `map`, `mode`, `played_at`, `duration`, `winner`.
6. Worker INSERT INTO `replays ... ON CONFLICT (fingerprint) DO NOTHING RETURNING id`.
  - Inserted: copy object to `replays/{sha256}.w3g`, insert `replay_players`, mark upload `accepted`.
  - Conflict: link upload to existing replay as a contributor, delete staging object, mark `duplicate`.
7. Worker DEL `list:latest` in Redis, acks SQS message. If the worker dies between 6 and 7 → SQS redelivers; step 6 is idempotent, step 7 re-runs. Cache stays stale ≤ TTL in the worst case.

### Read path (numbered)

1. Browser GET `/v1/replays?limit=50` → CDN. Hit (≤30 s old) → return. Otherwise forward.
2. API GET `list:latest` from Redis (a sorted JSON array of the last ~2K replays). Hit → slice by cursor, return.
3. Miss → single-flight lock (SET `lock:list NX PX 2000`), query Postgres ORDER BY `played_at DESC, id DESC LIMIT 2000`, SET `list:latest EX 60`, return. Other requests waiting on the lock retry after 50 ms.

4. Cursor beyond the cached window → keyset query straight to a read replica (rare; deep pages).
5. GET /v1/replays/{id}/download → 302 to https://cdn/replays/{sha256}.w3g; download counter incremented via Redis INCR, flushed to Postgres every 60 s (approximate is fine).

## 6. API

```
GET /v1/replays?limit=50&cursor=<opaque>&mode=1v1&player=<id> → {items:[...], next_cursor}
GET /v1/replays/{id} → metadata + players + download_url
GET /v1/replays/{id}/download → 302 → CDN
POST /v1/uploads (Idempotency-Key) {sha256,size_bytes} → {upload_id, put_url, expires_at}
POST /v1/uploads/{id}/complete → 202 {status:"queued"}
GET /v1/uploads/{id} → {status:
queued|accepted|duplicate|rejected, replay_id?}
```

- Cursor = base64(played\_at, id) of the last item. Opaque to clients; server can change encoding freely.
- Auth: reads are public; uploads require a player token or ladder-system API key. Uploader must appear in the parsed player list or be the system key — prevents random users injecting replays.
- Internal contract: ParseReplay{upload\_id, sha256, staging\_key, v:1} — versioned so parser changes don't break in-flight messages.

## 7. Data model (Postgres)

```
replays 1—N replay_players N—1 players
replays 1—N replay_uploads (audit / idempotency / contributors)
```

```
CREATE TABLE replays (
  id TEXT PRIMARY KEY, -- ULID, public, time-sortable
  fingerprint CHAR(64) NOT NULL UNIQUE, -- game identity (see §8)
  sha256 CHAR(64) NOT NULL, -- first accepted file's content hash
  mode TEXT NOT NULL, -- '1v1', '2v2', '3v3', '4v4'
  map_id INT NOT NULL,
  played_at TIMESTAMPTZ NOT NULL, -- from replay header, not upload time
  duration_s INT NOT NULL,
  game_version TEXT NOT NULL,
  winner_team SMALLINT,
  size_bytes INT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX replays_feed ON replays (played_at DESC, id DESC); -- the listing
CREATE INDEX replays_mode_feed ON replays (mode, played_at DESC, id DESC);

CREATE TABLE replay_players (
  replay_id TEXT REFERENCES replays(id), slot SMALLINT, player_id INT NOT NULL,
  team SMALLINT NOT NULL, race CHAR(1) NOT NULL, is_winner BOOL NOT NULL,
  PRIMARY KEY (replay_id, slot)
);
CREATE INDEX replay_players_by_player ON replay_players (player_id, replay_id);

CREATE TABLE replay_uploads (
  id TEXT PRIMARY KEY, uploader_id INT NOT NULL, sha256 CHAR(64) NOT NULL,
  replay_id TEXT REFERENCES replays(id),
  status TEXT NOT NULL, -- queued|accepted|duplicate|rejected
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

- Why Postgres, not DynamoDB: 1K writes/day, relational queries (player pages, map pages), one unique constraint doing the dedup. DynamoDB buys nothing here and costs the ON CONFLICT simplicity.
- Sort by played\_at (from header), not created\_at: a replay uploaded 3 days late belongs to the day it was played. Defeater: a client with a wrong clock can push a game into the future — clamp played\_at <= now() +

5 min at parse time.

- Hot-partition risk: none at this scale; at 100× write volume, partition `replays` by month (range partition on `played_at`).

## 8. Deep dive A — replay identity & dedup

8 clients → 8 uploads → parse header → same fingerprint → 1 row (unique constraint)

- Intuition: the replay ID must identify the **game**, not the **file**, because one game yields up to 8 files.
- Two candidate keys:
  - `sha256(file)`: correct **only if** every client writes byte-identical bytes. If the header embeds the saving player's slot or timestamp, the 8 files differ and you store 8 "replays".
  - `fingerprint = sha256(map_hash || game_version || random_seed || sorted(player names, slots) || played_at bucketed to 1 s)`: derived from parsed header, identical across clients.
  - ◆ Defeater: two games with the same players, map, seed, and second — practically impossible; the random seed is the discriminator.
  - ◆ Defeater: parser bug in one version yields a different fingerprint → duplicates appear. Version the fingerprint algorithm (`fp_v`) and backfill on change.
- How it is computed (Java 17):

```
// fields come from the .w3g header blocks (see w3g_format spec / w3gjs parser)
record GameHeader(int version, int build, String mapPath, long randomSeed,
                  List<Slot> slots) {}
record Slot(int slotId, String playerName, int team, int race) {}

static String fingerprint(GameHeader h) {
    String canonical = String.join("|",
        "fp_v1", // algorithm version
        Integer.toString(h.version()),
        Integer.toString(h.build()),
        h.mapPath().toLowerCase(),
        Long.toString(h.randomSeed()), // the real discriminator
        h.slots().stream()
            .sorted(Comparator.comparingInt(Slot::slotId))
            .map(s -> s.slotId() + ":" + s.playerName().toLowerCase() + ":" + s.team())
            .collect(Collectors.joining(",")));
    return HexFormat.of().formatHex(
        MessageDigest.getInstance("SHA-256").digest(canonical.getBytes(UTF_8)));
}
```

- Include only fields every client sees identically: version, map, random seed, slot/player/team.
- Exclude: replay length (a player who leaves early saves a shorter file), saving player's slot, upload time.
- The `.w3g` header carries no wall-clock timestamp (verify against spec) — so `played_at` comes from the uploader's `complete()` call or the ladder system, and is **not** part of the key.

- Decision: **public id = ULID, uniqueness = fingerprint, content hash = storage key**.
  - ULID gives clean, time-sortable URLs; the 64-char fingerprint is an internal constraint.
  - 🗣️: "The ID is not the hash. The hash is how I know two uploads are the same game."
- Race: two of the 8 uploads parsed concurrently on different workers.
  1. Both compute the same fingerprint.
  2. Both run `INSERT ... ON CONFLICT (fingerprint) DO NOTHING RETURNING id`.
  3. Postgres serializes on the unique index; exactly one gets a row back.
  4. The loser runs `SELECT id FROM replays WHERE fingerprint = ?`, records itself in `replay_uploads` as `duplicate`, deletes its staging object.
  - No locks, no Redis, no coordination. The unique index **is** the coordinator.

- Which file do we keep? The first accepted. If files differ per client, optionally keep all under `replays/{replay_id}/{slot}.w3g` — only if the product needs per-perspective replays. Default: keep one.
- **L5:** the 4v4 case is where a naive "hash the file and dedup" silently produces 8 rows and nobody notices until the feed looks spammy. The guardrail is a parse-time test fixture: 8 real client files of one game must collapse to one fingerprint in CI.

## 9. Deep dive B — the "today" listing, pagination, cache

Postgres (played\_at DESC, id DESC) → Redis list:latest (top 2000) → CDN (30 s) → client groups by day

- Intuition: "show today, backfill from yesterday" is just "newest first, always". The day boundary is presentation.
  - Server returns a continuous descending stream; the client inserts "Today / Yesterday" headers using `played_at` in the **user's** timezone.
  - Defeater of doing it server-side: "today" differs per timezone → cache fragments per timezone → cache hit rate collapses. Keep the server timezone-agnostic.
- Pagination: **keyset**, not offset.
  - `WHERE (played_at, id) < (:cursor_ts, :cursor_id) ORDER BY played_at DESC, id DESC LIMIT 50` — uses `replays_feed` index,  $O(\text{page})$  regardless of depth.
  - Offset defeater: a new replay inserted while paging shifts every offset → duplicated/skipped items on page 2. Also `OFFSET 100000` scans 100K rows.
  - `id` in the tuple breaks ties when two games end in the same second.
- Cache layout:
  - `list:latest` = one JSON array of the newest ~2000 replays ( $\approx 1$  MB). Any page inside the window is an in-memory slice. Deep pages (rare) go to a replica.
  - Alternative: cache per page (`list:p1`, `list:p2`...) — cheaper per key, but invalidation touches N keys and page boundaries drift. Rejected at this write rate.
  - Invalidation: worker `DEL`s on every accepted insert ( $\sim 1\text{K}/\text{day}$  — fine). TTL 60 s as a safety net for missed invalidations.
- Hot key: 100M readers on one Redis key.
  - 10K QPS on a single key is within one Redis node's range (order of magnitude; verify against your instance) but it is a single point of contention.
  - Mitigations in order: (1) CDN caches the JSON for 30 s, so origin sees  $\sim 1$  req/30 s per edge/page combo; (2) in-process cache in the API (Caffeine, 5 s) absorbs bursts; (3) Redis read replicas if still hot.
- Stampede on invalidation: all API pods miss at once and hammer Postgres.
  - Single-flight lock (`SET lock:list NX PX 2000`); losers wait 50 ms and re-read. Alternative: worker *rebuilds* the key instead of deleting it (write-through), which removes the miss entirely — preferred once traffic justifies it.
- **L5:** the 30 s CDN TTL is the real consistency contract. State it out loud: "a new replay appears within  $\sim 30$  s for everyone; the DB is strongly consistent, the feed is eventually consistent by design." At  $10\times$  writes nothing changes; at  $100\times$  (100K/day) the 2000-window is  $< 1$  hour of data and deep pages hit the replica — that's when you shard the cache by day.

## 10. Failure modes

| Failure                                   | Trigger                              | Blast radius | Detection                                                                       | Mitigation                                       |
|-------------------------------------------|--------------------------------------|--------------|---------------------------------------------------------------------------------|--------------------------------------------------|
| Duplicate replays in feed                 | fingerprint algo change / parser bug | Feed quality | Daily job: <code>GROUP BY fingerprint HAVING count&gt;1 = 0</code> ; CI fixture | Version <code>fp_v</code> , backfill, merge rows |
| Stampede on <code>list:latest</code> miss | Cache delete + burst                 | Postgres CPU | Replica QPS spike on invalidate                                                 | Single-flight lock; write-through rebuild        |

| Failure                              | Trigger                                                  | Blast radius                          | Detection                                      | Mitigation                                                                              |
|--------------------------------------|----------------------------------------------------------|---------------------------------------|------------------------------------------------|-----------------------------------------------------------------------------------------|
| Orphaned staging objects             | Client never calls <code>complete</code>                 | S3 cost only                          | Bucket size metric                             | 24 h lifecycle rule on <code>staging/</code>                                            |
| Poison replay                        | Malformed / adversarial <code>.w3g</code> crashes parser | Parser worker restart loop            | SQS <code>ApproximateAgeOfOldestMessage</code> | Parse in try/catch with size cap; <code>maxReceiveCount=3</code> → DLQ; reject          |
| Clock skew on <code>played_at</code> | Client with wrong clock                                  | Replay pinned to top of feed for days | Alert on <code>played_at &gt; now()</code>     | Clamp at parse; fall back to upload time                                                |
| CDN origin down                      | API/Redis outage                                         | Reads                                 | 5xx at edge                                    | CDN <code>stale-if-error</code> serves last good list; downloads unaffected (S3 direct) |
| Wrong uploader                       | Random user uploads someone else's game                  | Feed integrity                        | Uploader $\notin$ player list                  | Reject unless uploader in players or system key                                         |

- Retryable: S3 upload, SQS enqueue, Redis ops, parser transient I/O.
- Terminal: bad magic bytes, unsupported version, uploader not in game — surface as `rejected` with reason.

## 11. Pop-up questions

### Q: Why not just hash the file for dedup?

L4: works only if all clients write identical bytes; I don't assume that, so I fingerprint the parsed header. L5: and I'd verify empirically with 8 client files from one 4v4, then keep content hash as the storage key either way — it makes objects immutable and CDN-cacheable forever.

### Q: Why is the day boundary on the client?

L4: timezone. L5: server-side day grouping fragments the cache per timezone; a timezone-agnostic descending stream keeps one cache entry for all 100M users.

### Q: What if page 1 changes while I'm on page 3?

Keyset cursor pins position to `(played_at, id)`; new items appear above, never shift me. Offset would duplicate/skip.

### Q: Why Postgres and not DynamoDB?

1K writes/day; the unique constraint gives dedup for free; player/map queries are relational. Dynamo would need a conditional put plus GSIs to do the same thing with less clarity.

### Q: Isn't `list:latest` a single hot key?

Yes. CDN takes 99%+ of reads at a 30 s TTL; in-process cache takes bursts; replicas after that. I'd rather one well-shielded hot key than N keys with drifting page boundaries.

### Q: What happens if the parser worker dies mid-insert?

SQS visibility timeout expires → redelivered. `ON CONFLICT DO NOTHING` makes the insert idempotent; the S3 copy is idempotent by key; cache delete is idempotent.

### Q: 4v4 — 8 uploads at once. Anything break?

Only S3 staging cost (8 MB, deleted within minutes) and 8 parse jobs. The unique index resolves the winner; no lock needed.

### Q: Egress cost at 10 TB/day?

That's the bill. CDN with long TTL on immutable objects; consider tiering to a cheaper CDN/S3 transfer plan; optionally gzip (`.w3g` is already compressed internally — verify gain before doing it).

### Q: What changes at 10×?

Reads: nothing (CDN). Writes: 10K/day — still nothing. Storage: 10 GB/day — still nothing. At 100× writes,

partition replays by month and shard the list cache by day.

## 12. Tradeoffs & wrap-up

| Decision     | Chosen             | Alternative       | Flips when                                 |
|--------------|--------------------|-------------------|--------------------------------------------|
| Dedup key    | Header fingerprint | Content hash      | Files proven byte-identical across clients |
| Public ID    | ULID               | Fingerprint hex   | Never — URLs matter                        |
| Pagination   | Keyset             | Offset            | Never for a live feed                      |
| List cache   | One 2K-item blob   | Per-page keys     | Write rate >> 100K/day                     |
| Upload       | Presigned S3 PUT   | Proxy through API | Need synchronous virus scan / small files  |
| Parsing      | Async via SQS      | Inline in request | Files < 100 KB and parse < 100 ms          |
| Day grouping | Client             | Server            | Single-timezone product                    |

Defeaters (framings to reject):

- "Shard the DB" — 1K writes/day; sharding is cost with no benefit.
- "Kafka for uploads" — SQS with DLQ is sufficient; no replay/fan-out need.
- "Store replays in Postgres bytea" — 10 TB/day egress must come from CDN, not a DB.

🔑 **Wrap-up (2 min):** "The crux is a shared, nearly static feed read by 100M people and a trickle of writes that arrive 2–8 times each. I keep the truth in Postgres with a unique game fingerprint, bytes in S3 behind a CDN, and one Redis blob for the feed with keyset cursors. MVP0 is upload → parse → insert → list, no cache. Week one I watch cache hit rate, duplicate-fingerprint count, and CDN egress. At 10× nothing changes; at 100× I partition by month and shard the feed cache by day."

## 13. References

- PBug90/w3gjs — open-source .w3g parser (TypeScript); use it to confirm which header fields are stable across clients. [github.com/PSBug90/w3gjs](https://github.com/PSBug90/w3gjs) (verify link)
- Use The Index, Luke — "No Offset": keyset pagination rationale. [use-the-index-luke.com/no-offset](https://use-the-index-luke.com/no-offset)
- AWS docs — Uploading objects with presigned URLs. [docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html](https://docs.aws.amazon.com/AmazonS3/latest/userguide/PresignedUrlUploadObject.html)
- PostgreSQL docs — INSERT ... ON CONFLICT (upsert semantics under concurrency). [postgresql.org/docs/current/sql-insert.html](https://postgresql.org/docs/current/sql-insert.html)